gsb gsbfrais
BTS SIO 1

SLAM

02/09/26

AP-GSB

Sommaire :

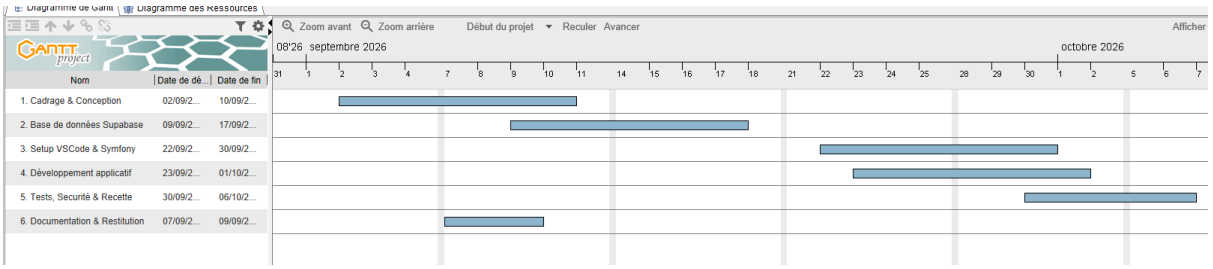
Contexte du projet:
Gant du projet :
Cahier des charges :
Charte Graphique GSB :
Diagramme de classe :

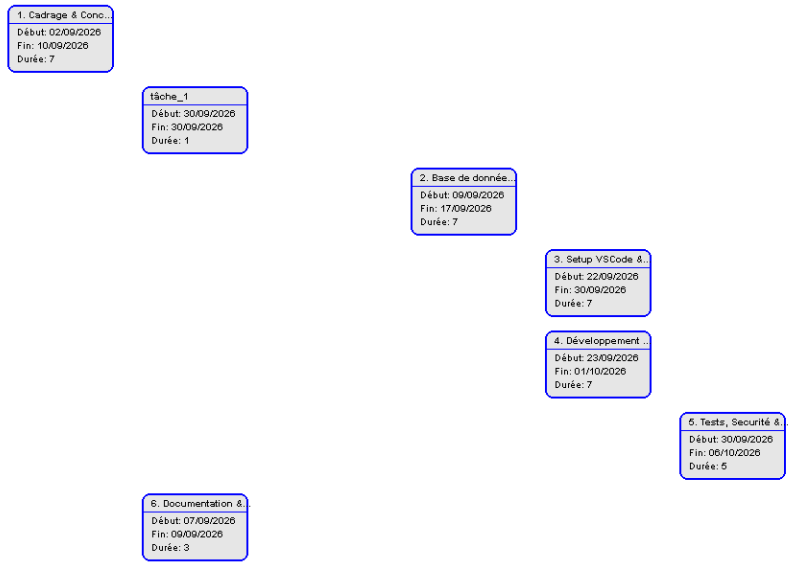
Contexte du projet:

Dans le cadre du laboratoire **GSB**, l'objectif de ce TP est de développer et finaliser une **application web de gestion des frais de déplacement**.

Cette solution vise à automatiser le suivi des dépenses des **visiteurs médicaux** (saisie des frais et envoi des justificatifs) et à permettre au **service comptable** de vérifier, valider et mettre en paiement ces demandes de remboursement de manière sécurisée.

Gant du projet :





Cahier des charges :

1. **Objectif**
Centraliser et automatiser la gestion, la validation et le remboursement des frais de déplacement du Laboratoire GSB.

2. **Design & Ergonomie**

- **Palette :** Bleu foncé (en-tête), Bleu roi (boutons principaux), Rouge (suppression), Fond gris clair/blanc.
-
- **Structure :** En-tête fixe (infos/déconnexion), onglets horizontaux, tableaux et formulaires sous forme de cartes blanches.
-

3**. Fonctionnalités par Rôle**

Visiteur Médical

- **Ma fiche du mois :**
-
    - Choix du véhicule (calcul auto du barème).
    -
    - Saisie des frais forfaitisés (nuit, repas, km).
    -
    - Calculateur de distance pour les trajets (ville A → ville B).
    -
    - Saisie des frais hors forfait avec justificatif obligatoire.
    -
    - Export PDF de la fiche.
    -
- **Mes fiches précédentes :** Historique annuel et suivi des états (*En saisie, Clôturée, Validée, Remboursée*).
-

Service Comptable

- **Validation :** Contrôle des montants et justificatifs, validation/refus des lignes hors forfait, validation globale.
-
- **Mise en paiement :** Traitement des fiches à régler (virements le 20 du mois) et suivi jusqu'à remboursement.
-
- **Suivi & Exports :** Vue globale par statut et génération de PDF (individuels ou groupés par mois).
-

Administrateur

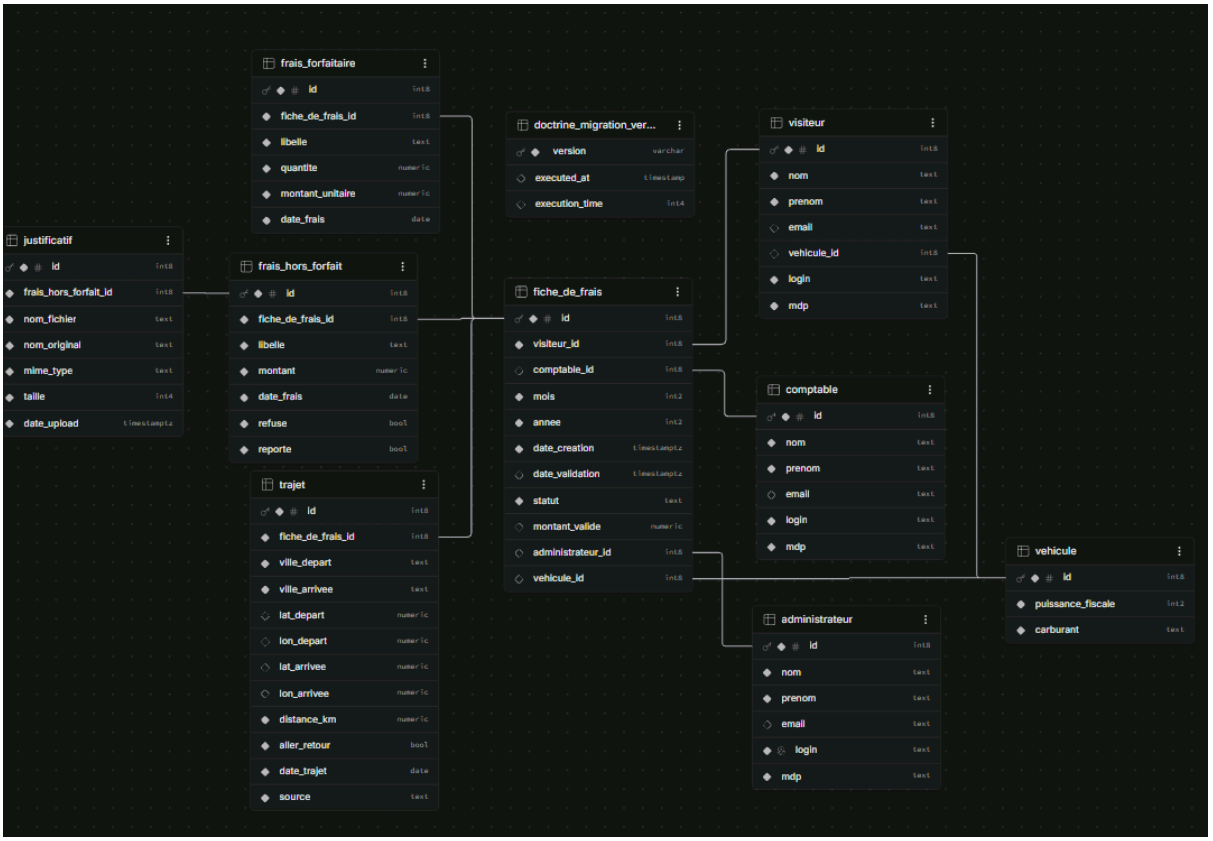
- **Gestion des accès :** Création, modification et suppression des comptes *Visiteurs* et *Comptables*.

:

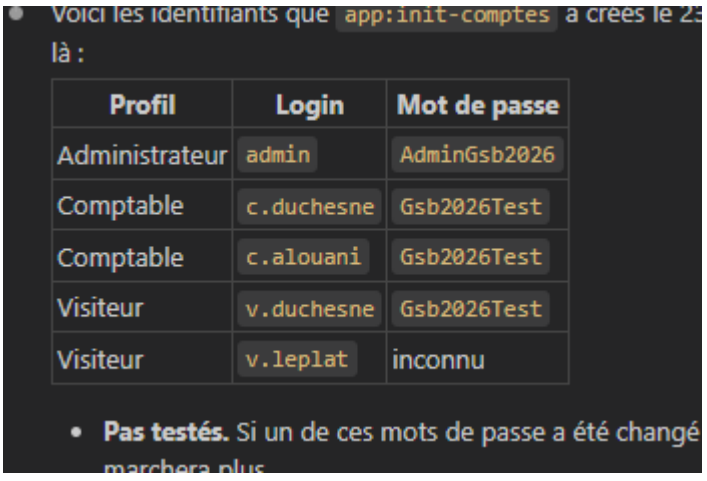
Charte Graphique GSB :

- Couleurs principales :
-
    - Bleu foncé / Bleu nuit : Barre d'en-tête (header) et navigation principale.
    -
    - Bleu roi : Boutons d'action principaux (Se connecter, Enregistrer, Ajouter, Créer).
    -
- Couleur d'alerte / Action :
-
    - Rouge : Boutons de suppression (Supprimer).
    -
- Fond et Conteneurs :
-
    - Fond de page : Blanc / Gris très clair.
    -
    - Cartes & Tableaux : Bloc blancs sur-élevés ou délimités par une fine bordure grise.
    -
- Typographie :
-
    - Police Sans-Serif (propre, moderne et lisible).
    -
- Structure (Layout) :
-
    - En-tête supérieur : Titre, nom/rôle de l'utilisateur connecté et bouton Déconnexion.
    -
    - Sous-menu : Onglets de navigation horizontaux.

Diagramme de classe :



LOGIN :



Voici les identifiants que `app:init-comptes` a créés le 2
là :

Profil	Login	Mot de passe
Administrateur	`admin`	`AdminGsb2026`
Comptable	`c.duchesne`	`Gsb2026Test`
Comptable	`c.alouani`	`Gsb2026Test`
Visiteur	`v.duchesne`	`Gsb2026Test`
Visiteur	`v.leplat`	inconnu

- **Pas testés.** Si un de ces mots de passe a été changé
marchera plus

c« On génère le PDF avec Dompdf, une bibliothèque PHP qui convertit une page HTML en PDF. On crée le HTML avec un gabarit Twig, comme les autres pages du site, et Dompdf le met en page au format A4.

GUIDE DES FICHIERS :

Les fichiers du projet GSB Frais

Du plus important au moins important.

---

1. Le code de l'application (à connaître pour l'oral)

Les données — `src/Entity/`

- **FicheDeFrais.php** → la fiche d'un visiteur pour un mois : frais, trajets, véhicule, état et totaux.
- **Visiteur.php** → un visiteur médical, qui se connecte pour saisir ses frais.
- **Comptable.php** → un comptable, qui valide les fiches et les met en paiement.
- **Administrateur.php** → l'administrateur, qui gère les comptes et traite aussi les fiches.
- **FraisForfaitaire.php** → une ligne au forfait : nuitée, repas, étape ou kilomètres.
- **FraisHorsForfait.php** → une dépense réelle (libellé, date, montant), qui peut être refusée.
- **Trajet.php** → un déplacement d'une ville à une autre, avec sa distance calculée.
- **Vehicule.php** → une puissance fiscale + un carburant ; calcule le tarif au kilomètre.
- **Justificatif.php** → les informations d'une facture déposée (le fichier est sur le disque).
- **Statut.php** → les 5 états d'une fiche : créée, clôturée, validée, mise en paiement, remboursée.

Les pages et les actions — `src/Controller/`

- **VisiteurController.php** → tout ce que fait le visiteur : sa fiche, ses frais, ses trajets, ses factures, son PDF.
- **ComptableController.php** → tout ce que fait le comptable : valider, refuser, mettre en paiement, imprimer.
- **AdminController.php** → la gestion des comptes : créer, changer le mot de passe, supprimer.
- **SecurityController.php** → la page de connexion et la déconnexion.
- **AccueilController.php** → envoie chaque utilisateur vers son espace selon son rôle.

Les règles métier — `src/Service/`

- **GestionnaireFiche.php** → les règles des fiches : création du mois, clôture, report des frais refusés, validation.

- **CalculateurDistance.php** → calcule la distance entre deux villes avec deux services en ligne.e
- **GenerateurPdf.php** → transforme une fiche en PDF.
- **StockageJustificatif.php** → enregistre les factures déposées et vérifie leur format et leur taille.
- **Lieu.php** → une commune avec ses coordonnées GPS.
- **ResultatDistance.php** → le résultat d'un calcul de distance.
- **VilleIntrouvableException.php** → l'erreur affichée quand une ville n'existe pas.

Les pages HTML — `templates/`

- **base.html.twig** → le modèle commun à toutes les pages : bandeau, menu, messages.
- **visiteur/fiche.html.twig** → la fiche du mois du visiteur.
- **visiteur/historique.html.twig** → la liste des fiches passées du visiteur.
- **comptable/selection.html.twig** → le choix de la fiche à valider.
- **comptable/validation.html.twig** → le contrôle et la validation d'une fiche.
- **comptable/paiement.html.twig** → la mise en paiement des fiches validées.
- **comptable/suivi.html.twig** → toutes les fiches, rangées par état.
- **comptable/editions.html.twig** → les impressions PDF du comptable.
- **admin/comptes.html.twig** → la page de gestion des comptes.
- **security/login.html.twig** → le formulaire de connexion.
- **pdf/fiche.html.twig** → le modèle du PDF imprimé.

Les requêtes en base — `src/Repository/`

- **FicheDeFraisRepository.php** → les recherches de fiches (par visiteur, par mois, par état…).
- **VisiteurRepository.php** → les recherches de visiteurs.
- **ComptableRepository.php** → les recherches de comptables.
- **AdministrateurRepository.php** → vérifie si un identifiant est déjà pris.
- **VehiculeRepository.php** → la liste des 6 véhicules du barème.
- **FraisForfaitaireRepository.php**, **FraisHorsForfaitRepository.php**, **JustificatifRepository.php**, **TrajetRepository.php** → vides, créés avec leur entité.

Les modifications de la base — `migrations/`

- **Version20260923000001.php** → ajoute la connexion, l'administrateur, les justificatifs et les 5 états.
- **Version20260923000002.php** → rend les identifiants obligatoires et uniques.
- **Version20260923000003.php** → remet à jour les compteurs d'identifiants.
- **Version20260923000004.php** → ajoute les trajets et la validation par l'administrateur.
- **Version20260923000005.php** → met le véhicule sur la fiche.
- **Version20260930000001.php** → rend le véhicule du compte facultatif.

Les commandes — `src/Command/`

- **InitComptesCommand.php** → `app:init-comptes` : crée les identifiants et le compte administrateur.

- **CloturerFichesCommand.php** → `app:cloturer-fiches` : clôture les fiches des mois passés.
- **JeuDeDonneesCommand.php** → `app:jeu-de-donnees` : crée des fiches de test.

---

2. Les autres fichiers du projet

- **public/css/gsb.css** → l'apparence de toutes les pages.
- **public/js/formulaires.js** → empêche d'envoyer deux fois un formulaire.
- **public/maquette/validation.html** → la maquette de l'écran de validation demandée par l'énoncé.
- **README.md** → la présentation du projet et son installation.
- **docs/schema_supabase.sql** → ⚠ ancien schéma de la base, qui ne correspond plus à la vraie.
- **docs/guide-des-fichiers.md** → ce guide.
- **docs/calcul-des-distances.md** → l'explication du calcul automatique des distances.
- **config/packages/security.yaml** → qui peut se connecter et à quelles pages chacun a accès.
- **config/packages/doctrine.yaml** → la connexion à la base de données.
- **.env.local** → ⚠ l'adresse et le mot de passe de la base Supabase, à ne jamais partager.
- **.env** → les réglages par défaut de l'application.
- **.env.dev** → la clé secrète pour le développement.
- **.env.test** → les réglages pour les tests (non utilisés).

---

3. Les fichiers de Symfony (on n'y touche presque jamais)

- **public/index.php** → la porte d'entrée : toutes les pages passent par lui.
- **src/Kernel.php** → le cœur de Symfony, qui démarre l'application.
- **bin/console** → l'outil pour lancer les commandes (`php bin/console …`).
- **composer.json** → la liste des bibliothèques utilisées par le projet.
- **composer.lock** → les versions exactes de ces bibliothèques.
- **symfony.lock** → la liste des configurations installées automatiquement par Symfony.
- **config/bundles.php** → la liste des modules Symfony activés.
- **config/services.yaml** → permet d'utiliser les classes de `src/` partout sans configuration.
- **config/routes.yaml** → dit à Symfony de trouver les adresses des pages dans les contrôleurs.
- **config/routes/framework.yaml** → les pages d'erreur en développement.
- **config/routes/security.yaml** → l'adresse de déconnexion.
- **config/routes/web_profiler.yaml** → l'adresse de la barre de débogage.
- **config/packages/framework.yaml** → les réglages de base de Symfony.
- **config/packages/csrf.yaml** → la protection des formulaires contre les attaques.
- **config/packages/twig.yaml** → les réglages des pages HTML.
- **config/packages/validator.yaml** → active la vérification des saisies.

- **config/packages/cache.yaml** → le cache, qui garde en mémoire les distances déjà calculées.
- **config/packages/monolog.yaml** → l'écriture du journal de l'application.
- **config/packages/routing.yaml** → l'adresse de base du site.
- **config/packages/doctrine_migrations.yaml** → l'emplacement des migrations.
- **config/packages/debug.yaml** → les outils de débogage.
- **config/packages/web_profiler.yaml** → la barre de débogage en bas des pages.
- **config/packages/property_info.yaml** → un réglage par défaut de Symfony.
- **config/packages/translation.yaml** → la traduction (non utilisée, l'appli est en français).
- **config/preload.php** → une optimisation pour la mise en production.
- **config/reference.php** → la liste de toutes les options de configuration, générée automatiquement.
- **.gitignore** → les fichiers que Git ne doit pas enregistrer.
- **.editorconfig** → les règles de mise en forme pour l'éditeur de code.

---

4. Les fichiers installés mais inutilisés

- **assets/app.js** → le JavaScript de Symfony, jamais chargé par nos pages.
- **assets/stimulus_bootstrap.js** → démarre Stimulus (non utilisé).
- **assets/controllers.json** → la configuration de Stimulus (non utilisée).
- **assets/controllers/hello_controller.js** → un exemple fourni par Symfony.
- **assets/controllers/csrf_protection_controller.js** → un exemple fourni par Symfony.
- **assets/styles/app.css** → une feuille de style d'exemple (non utilisée).
- **assets/vendor/@hotwired/stimulus/stimulus.index.js** → la bibliothèque Stimulus (non utilisée).
- **assets/vendor/@hotwired/turbo/turbo.index.js** → la bibliothèque Turbo (non utilisée).
- **assets/vendor/installed.php** → la liste de ces bibliothèques.
- **importmap.php** → la liste des fichiers JavaScript (non utilisée).
- **config/packages/asset_mapper.yaml** → la gestion du dossier `assets/` (non utilisée).
- **config/packages/ux_turbo.yaml** → la configuration de Turbo (non utilisée).
- **config/packages/mailer.yaml** → l'envoi de mails (non utilisé).
- **config/packages/messenger.yaml** → les tâches en arrière-plan (non utilisées).
- **config/packages/notifier.yaml** → les notifications (non utilisées).
- **compose.yaml** → une base de données Docker (non utilisée, la base est sur Supabase).
- **compose.override.yaml** → des réglages Docker (non utilisés).
- **src/DataFixtures/AppFixtures.php** → des données de test, vide. ⚠ Ne jamais lancer `doctrine:fixtures:load`, qui vide toute la base.
- **phpunit.dist.xml** → la configuration des tests automatiques (aucun test écrit).
- **bin/phpunit** → lance les tests automatiques.
- **tests/bootstrap.php** → prépare les tests automatiques.
- **translations/** → dossier des traductions, vide.
- **src/Controller/.gitignore**, **src/Entity/.gitignore**, **src/Repository/.gitignore**, **migrations/.gitignore**, **translations/.gitignore** → fichiers vides qui gardent les dossiers dans Git.

---

5. Les dossiers créés automatiquement (ne jamais modifier)

- **vendor/** → le code des bibliothèques (Symfony, Doctrine, Twig, Dompdf…), recréé par `composer install`.
- **var/cache/** → les fichiers temporaires de Symfony, recréés tout seuls.
- **var/log/** → le journal de tout ce qui se passe dans l'application.
- **var/justificatifs/** → les factures déposées par les visiteurs.
- **var/share/** → un dossier interne de Symfony, vide.

EXEMPLE DE FONCTIONNALITE :

Le calcul automatique des distances

Le visiteur ne tape pas de kilomètres : il donne deux villes, et l'application trouve elle-même la distance par la route. Ce document explique comment, en détail d'abord, puis en version simple.

---

1. Explication détaillée

Les fichiers concernés

- **templates/visiteur/fiche.html.twig** → le formulaire « Ajouter un trajet ».
- **src/Controller/VisiteurController.php** → reçoit le formulaire (méthode `ajouterTrajet()`).
- **src/Service/CalculateurDistance.php** → fait le calcul.
- **src/Service/Lieu.php** → une ville avec ses coordonnées GPS.
- **src/Service/ResultatDistance.php** → le résultat du calcul.
- **src/Service/VilleIntrouvableException.php** → l'erreur « ville introuvable ».
- **src/Entity/Trajet.php** → le trajet enregistré en base.
- **src/Service/GestionnaireFiche.php** → reporte les kilomètres sur la fiche (`synchroniserKilometrage()`).

Étape 1 : le visiteur remplit le formulaire

Sur sa fiche du mois, le visiteur saisit une date, une ville de départ, une ville d'arrivée, et coche « aller-retour » si besoin. Puis il clique sur « Calculer et ajouter ».

Le formulaire n'apparaît que si la fiche est modifiable et qu'un véhicule est déclaré : sans véhicule, il n'y a pas de tarif kilométrique.

Étape 2 : le contrôleur vérifie la demande

`VisiteurController::ajouterTrajet()` reçoit le formulaire. Avant tout calcul, il vérifie :

1. que la fiche appartient bien au visiteur connecté ;
2. qu'elle est encore en saisie ;
3. que le jeton de sécurité du formulaire (CSRF) est valide ;
4. qu'un véhicule est déclaré ;
5. que la date est valide.

Il appelle ensuite le calculateur :

```php
$resultat = $calculateur -> calculer(depart, $arrivee);
```

Étape 3 : transformer chaque ville en coordonnées GPS (géocodage)

Méthode `geocoder()` de `CalculateurDistance`.

Un ordinateur ne sait pas où se trouve « Lyon ». Il faut d'abord convertir le nom en coordonnées (latitude, longitude). Pour ça, l'application interroge la **Base Adresse Nationale**, un service public gratuit de l'État, sans clé d'accès :

```
https://api-adresse.data.gouv.fr/search/?q=Lyon&type=municipality&limit=1
```

- `q=Lyon` : la ville cherchée ;
- `type=municipality` : on cherche une commune, pas une rue ;
- `limit=1` : on ne garde que le meilleur résultat.

Le service répond en JSON. L'application en garde le nom officiel de la commune, son code postal, sa latitude et sa longitude, et les range dans un objet `Lieu`.

Si le service ne trouve rien, le calculateur lève une `VilleIntrouvableException`. Le contrôleur la transforme en message pour le visiteur : « Commune « Lyonn » introuvable. Vérifiez l'orthographe de la commune. » Aucun trajet n'est alors enregistré.

Le géocodage est fait deux fois : une pour la ville de départ, une pour la ville d'arrivée.

Étape 4 : demander la distance par la route (routage)

Méthodes `distanceRoutiere()` et `interrogerRoutage()`.

Avec les deux points GPS, l'application interroge **OSRM** (Open Source Routing Machine), un calculateur d'itinéraire gratuit basé sur les cartes OpenStreetMap :

```
https://router.project-osrm.org/route/v1/driving/{longitude départ},{latitude départ};{longitude arrivée},{latitude arrivée}
```

- `driving` : un itinéraire en voiture ;
- la réponse contient la longueur du meilleur itinéraire, **en mètres**.

L'application divise par 1 000 pour obtenir des kilomètres, arrondis à deux décimales. Exemple réel : Paris → Lyon = **465,42 km**.

Étape 5 : le plan B si OSRM ne répond pas (vol d'oiseau)

Méthode `orthodromie()`.

OSRM est un service gratuit : il peut être lent ou en panne. L'application attend 12 secondes au maximum. Si elle n'a pas de réponse valable, elle ne bloque pas le visiteur : elle calcule la distance **à vol d'oiseau** entre les deux points GPS, avec la **formule de haversine**, qui tient compte de la forme ronde de la Terre (rayon de 6 371 km) :

```
h = sin²(Δlatitude / 2) + cos(latitude1) × cos(latitude2) × sin²(Δlongitude / 2)
distance = 2 × 6 371 × arcsin(√h)
```

Une route n'est jamais en ligne droite. Le résultat est donc multiplié par

1,3 (constante `COEFFICIENT_ROUTIER`) pour approcher la distance réelle.

Le visiteur est prévenu par un message orange : « Le service de calcul d'itinéraire était indisponible : la distance retenue est une estimation à vol d'oiseau majorée. »

Étape 6 : le cache, pour ne pas recalculer

Les visiteurs refont souvent les mêmes trajets. Chaque résultat (coordonnées d'une ville, distance entre deux points) est donc gardé dans le **cache** de Symfony, dans le dossier `var/cache/`. La deuxième fois qu'on demande Paris → Lyon, la réponse est immédiate et aucun service extérieur n'est interrogé.

Étape 7 : enregistrer le trajet

De retour dans le contrôleur, un objet `Trajet` est créé et enregistré en base avec :

- les noms officiels des deux communes ;
- leurs coordonnées GPS ;
- la distance d'un aller simple ;
- la case aller-retour ;
- la date ;
- la **source** du calcul : `osrm` (route réelle) ou `haversine` (vol d'oiseau).

Les coordonnées et la source sont gardées pour pouvoir justifier une distance si elle est contestée. Le comptable voit la source sur son écran de validation, et elle figure aussi dans le détail affiché au visiteur.

Pour un aller-retour, `Trajet::getDistanceTotale()` double la distance : Paris → Lyon aller-retour = 465,42 × 2 = **930,84 km**.

Étape 8 : reporter les kilomètres sur la fiche

`GestionnaireFiche::synchroniserKilometrage()` :

1. additionne la distance totale de **tous** les trajets de la fiche ;
2. arrondit au kilomètre près ;
3. écrit ce nombre dans la ligne « Frais kilométrique » de la fiche ;
4. y applique le tarif du véhicule déclaré (par exemple 0,65 € par km pour un 7 CV diesel).

Si la fiche n'a plus aucun trajet, la ligne kilométrique est supprimée : la base

refuse une quantité égale à zéro.

Le visiteur voit enfin un message de confirmation, par exemple : « Trajet Paris → Lyon (aller-retour) enregistré : 930,84 km (distance routière calculée). Kilométrage de la fiche porté à 931 km. »

Étape 9 : la correction manuelle reste possible

La quantité kilométrique reste modifiable à la main dans le tableau des frais forfaitisés. En revanche, le prochain ajout ou la prochaine suppression de trajet remettra la somme calculée. Tant que les deux ne correspondent pas, l'écran le signale. Le comptable peut lui aussi corriger la quantité lors de la validation.

---

2. En version simple

Le visiteur écrit deux villes, par exemple Paris et Lyon. L'application demande d'abord à un service de l'État où se trouvent ces deux villes sur la carte : on obtient leurs coordonnées GPS. Elle demande ensuite à un calculateur d'itinéraire, comme un GPS de voiture, quelle est la longueur de la route entre ces deux points : 465 km. Si ce calculateur ne répond pas, l'application mesure la distance en ligne droite et ajoute 30 %, parce qu'une route fait toujours des détours.

La distance est enregistrée avec le trajet, doublée s'il s'agit d'un aller-retour. L'application additionne ensuite tous les trajets du mois et multiplie le total par le tarif du véhicule du visiteur : c'est le montant de ses frais kilométriques. Le visiteur n'a jamais à taper lui-même un nombre de kilomètres, ce qui évite les erreurs et les fraudes.